RTL Design & Functional Simulation (Verilog)

Goal: Design a synthesizable 8-bit ALU in Verilog and verify it using a comprehensive testbench.

Overview

The design implements an 8-bit ALU supporting addition, subtraction, bitwise AND, OR, XOR, and NOT operations. A synthesizable RTL module is written in Verilog and verified using a testbench containing both directed and random test cases.

RTL Source (alu8.v)

```
module alu(
input [7:0] A,
input [7:0] B,
input [2:0] opcode,
output reg [7:0] result
);
always @(*) begin
case(opcode)
3'b000: result = A + B;
3'b001: result = A - B;
3'b010: result = A & B;
3'b011: result = A | B;
3'b100: result = A ^ B;
default: result = 0;
endcase
end
endmodule
```

Testbench (tb_alu8.v)

```
`timescale 1ns/1ps
module alu_tb;
reg [7:0] A, B;
reg [2:0] opcode;
wire [7:0] result;
  alu uut (
.A(A),
.B(B),
.opcode(opcode),
.result(result)
);
initial begin
$dumpfile("dump.vcd");
$dumpvars(0, alu_tb);
A = 10; B = 5; opcode = 0;
#10;
A = 10; B = 5; opcode = 1;
#10;
A = 10; B = 5; opcode = 2;
#10;
A = 10; B = 5; opcode = 3;
#10;
A = 10; B = 5; opcode = 4;
#10;
$finish;
end
  endmodule
```

Simulation

Compile both files in ModelSim, QuestaSim, Icarus Verilog, or any Verilog simulator. Run the testbench and open the generated waveform (VCD/WLF) in GTKWave or ModelSim Wave window.

Coverage Explanation

Directed tests verify each ALU operation individually. Fifty random tests improve confidence by exercising different input combinations and operations. Functional coverage is achieved by ensuring every opcode (000-101) is executed at least once and both zero/non-zero outputs are observed.

Simulation Wave Screenshot